

Vehicle Detection and Classification for Smart Parking Management Using OpenCV-Based Computer Vision Techniques

Computer Science and Engineering
Lovely Professional University
Punjab, India

PALAK KUSHWAHA
(12217366)

KARTIK
(12217668)

Submitted to :- Kriti

Abstract

With the growing need for intelligent urban infrastructure, smart parking solutions are becoming critical for space efficiency and congestion management within Intelligent Transportation Systems (ITS). This paper proposes an OpenCV-based computer vision solution for real-time tracking, vehicle detection, and vehicle classification to automate parking management. Vehicles are detected and classified through background subtraction and contour detection to detect entry and exit events. The automation allows for a dynamic and efficient parking management platform that optimizes available capacity. The proposed solution is lightweight, accurate, and scalable to both public and private applications.

Keywords—Smart Parking System, Vehicle Detection, Vehicle Classification, OpenCV, Computer Vision, Real-Time Tracking, Background Subtraction, Contour Detection, Automated Parking Management, Intelligent Transportation Systems (ITS)

I. INTRODUCTION

As urbanization and the number of vehicles on the road increases, it is certainly a resulting fact that traffic congestion and inefficient use of parking space is a significant problem in metropolitan cities. Existing parking systems are mostly based on manual work or use a sensor-based approach which can either be unreliable or cost prohibitive for larger implementation. With these limitations, intelligent systems that make use of computer vision technologies to automate parking systems have been developed. The presented paper proposes detection and

classification system for vehicles using image processing techniques based entirely on OpenCV (Open Source Computer Vision Library). Differing from deep learned methods that require high powered hardware infrastructures and datasets to run with, this approach uses simple lightweight algorithms based on background subtraction, contour detection, and feature based syndrome classification which are able to process real-time detection of vehicles entering and exiting a parking lot without the need of specialized hardware, or GPU capabilities. This system will have the capability to receive live video input from recently-launched surveillance cameras placed at the entrances and exits of parking lot facilities. Upon streams of video input, vehicles will be detected using motion-detection based extraction, and then be classified based on a categories such as two-wheeled vehicles, cars, and heavy vehicles based on specific size, shape, and contour characteristics. In addition, simple tracking capabilities for the vehicle will be developed through use of a motion form of history logging as a tracking module then draw the tracked vehicle back on the original video.[1][2][5]

The exponential growth of urban population and the resulting increase in personal vehicle ownership have created serious issues that affect traffic management and usage of parking facilities. The lack of efficient parking in urban areas leads to vehicle congestion, carbon emissions, and driver dissatisfaction. Urban mobility research suggests many of the vehicles in cities are driven by drivers looking for parking, highlighting a critical need for innovative and automated parking management to improve parking and urban mobility. Traditional parking systems often rely on human operators, RFID tags, or sensor-based infrastructure to monitor and allocate the inflow of vehicles into their space. Although these systems are common, they have several limitations, including high initial and continued maintenance equipment costs, reduced scalability, and reduced responsiveness dependent on local conditions. Further, sensor-based systems often do not work reliably in conditions of reduced visibility, poor weather, and increased electronic noise. This paper proposes a low-cost, scalable, efficient smart parking solution using a standard data structure approach that is both inexpensive and builds from OpenCV-based computer

vision approaches. Unlike deep learning or IOT-based solutions, the proposed system does not require GPU hardware, expensive sensors[6][8][9]

The system supports various real-time operations, including:

- Allocating and deallocating parking slots
- Issuing parking tickets
- Retrieving slot numbers based on vehicle registration
- Querying vehicle data by driver age

These functionalities are accessed via structured command-line commands or input files, which gives the system a modular architecture and allows extensibility and integration into higher-level interfaces such as mobile or web-based applications. By taking advantage of the core programming capabilities of OpenCV and Python, the proposed solution offers a superior alternative to resource-heavy smart parking alternatives. The proposed system has the ambition of providing smart parking services with a lower reliance on infrastructure, which would allow it to be deployed in locations such as housing complexes, institutional campuses, and small-scale businesses.[4][10]

II. OPENCV FRAMEWORK AND ITS ROLE IN SMART PARKING

The Open Source Computer Vision Library (OpenCV) is an open-source software library for computer vision and machine learning, created to provide a single environment for image and video processing in real-time. OpenCV was originally developed by Intel and is currently supported by the open-source community. OpenCV contains over 2500 optimized algorithms with applications that include object detection, facial recognition, gesture recognition, motion tracking, and image classification. Within the proposed smart parking solution, OpenCV will be used to carry out some of the basic computer vision functionality required for detecting vehicle movement and the flow of vehicles in and out of the parking lot. OpenCV is lightweight and compatible with Python, which is perfect for creating a resource-efficient parking solution rather than implementing a heavyweight deep learning package.

A. Basis of OpenCV in Computer Vision

OpenCV is primarily built around the concept of processing images and videos by converting them into matrices and then applying operations such as:

- **Grayscale Conversion:** Simplifying the input image by converting it into grayscale, reducing computation.
- **Gaussian Blur and Noise Reduction:** Smoothing the image to minimize high-frequency noise that may lead to false detections.
- **Background Subtraction:** Detecting moving objects by identifying differences between the current frame and a static background reference.
- **Contour Detection:** Identifying object boundaries, which are then used to locate and segment vehicles within the frame.

- **Bounding Boxes and Object Tracking:**

Drawing rectangles around detected vehicles and monitoring their motion across frames.

These operations form the building blocks for detecting and identifying the presence of vehicles at entry and exit points of the parking lot.

B. OpenCV Applications in the Parking System

The implementation of OpenCV in this research is designed to:

1. **Detect Vehicle Presence:** When a car enters the parking lot, OpenCV captures and processes video frames using background subtraction to identify motion and potential vehicles.
2. **Trigger Slot Allocation:** Upon detecting a vehicle, the system triggers the `allocate_parking_slot()` method to assign the nearest available slot using a min-heap structure.
3. **Enable Real-Time Processing:** The fast processing capabilities of OpenCV enable near real-time response for detecting vehicle events without significant delay.
4. **Ensure Resource Efficiency:** By using OpenCV's built-in functions and avoiding complex models, the system remains lightweight and suitable for edge devices or low-power processors.

C. Advantages of Using OpenCV in the Research

- **Platform Independent:** OpenCV runs seamlessly across various platforms (Windows, Linux, macOS) and supports multiple languages (Python, C++, Java).
- **No Training Required:** Unlike deep learning models that require annotated datasets, OpenCV uses rule-based methods, eliminating the need for training and large storage.
- **Scalable and Adaptable:** The modular design of OpenCV-based functions allows easy expansion to incorporate features such as license plate recognition or vehicle type classification.

By incorporating OpenCV into the system architecture, this research offers a low-cost, scalable, and efficient approach to smart parking management suitable for medium to small-scale urban deployments.

OPENCV FRAMEWORK AND ITS ROLE IN SMART PARKING THROUGHOUT THE YEARS.



III. LITERATURE REVIEW

The development of smart parking systems has attracted considerable research interest due to increasing urbanization, the growing number of vehicles, and the need for efficient parking space management. Conventional parking methods often rely on human supervision or sensor-based solutions, which, while effective in small-scale environments, do not scale well in larger, dynamic urban environments.

In recent years, numerous smart systems have been developed that combine computer vision, the Internet of Things (IoT), and machine learning. For example, **S. Rizwan, F. Ahmed, and M. Z. Raja** [2] proposed a smart parking system using IoT sensors to detect vehicles and parking space availability. While sensor-based systems are effective, they often incur high installation and maintenance costs and are susceptible to environmental influences such as dust, rain, or temperature fluctuations.

A Real-Time Smart Parking System Using Image Processing is explained by **P. Kumar, R. R. Patra**[4]. A YOLO (You Only Look Once) model for vehicle detection in parking lots was trained using surveillance camera footage. This approach demonstrated high accuracy but required large labeled datasets and GPU resources for inference, making it less practical for low-cost real-time applications.

T. Nguyen, D. Nguyen, N. Duong [10] presented a vision-based parking guidance system that uses camera images to estimate parking space occupancy by analyzing pixel intensity changes. Their work inspired subsequent researchers to develop simpler models using background subtraction and contour detection to identify moving vehicles and vacant parking spaces.

Similarly, researchers using deep learning, **A. Aljumaily, H. H. Malik**, [5] investigated morphological filtering and edge detection techniques to track vehicle shapes and determine occupancy. Their results highlighted the potential of OpenCV-based methods for deployment on embedded systems.

More recently, hybrid systems have been developed that integrate Flask-based web interfaces with OpenCV to create user-friendly parking solutions. These models offer a responsive frontend and leverage classic vision algorithms such as ORB, Canny edge detection, and thresholding for backend processing.

Although advanced models such as CNNs and object trackers offer high detection performance, their reliance on labeled data and hardware accelerators makes them less ideal for cost-sensitive environments. In contrast, the proposed system leverages OpenCV's classic image processing capabilities, including grayscale transformation, Gaussian blur, histogram equalization, and real-time segmentation, thus providing a scalable and energy-efficient alternative.

Overall, the reviewed literature suggests a growing interest in developing accessible, real-time, and cost-effective parking management systems using computer vision. The proposed approach builds on this foundation, combining feature extraction, perspective correction, and web-based interaction to provide a practical solution for automated parking management.

IV. SYSTEM ARCHITECTURE

The proposed smart parking management system is designed using a modular and object-oriented architecture, combining computer vision with classical data structures to deliver real-time vehicle tracking and slot management. The core objective is to automate vehicle detection, record driver and vehicle information, and ensure optimal allocation of parking resources using an efficient algorithmic approach. The architecture is implemented in Python using the OpenCV library for computer vision and heap-based structures for slot management.

A. Vehicle Detection

The **Vehicle Detection** module is responsible for identifying and confirming the presence of vehicles the parking area. This detection process is executed using OpenCV, a high-performance computer vision library that provides the tools necessary for real-time image processing.

1) Working Principle

The module works by assessing dataset of continuous video streams from surveillance cameras located at critical positions. In our project the specially outfitted cameras will be located primarily at the entrance and exit of the parking area. openCV will treat the video stream as a continuous series of images whereby it processes it frame by frame and recognizes motion and identifies vehicles based on shape and contour characteristics.

Step-by-Step Process:

- **Frame Capture:**
OpenCV's `cv2.VideoCapture()` is used to continuously read video frames from a live feed or pre-recorded video input.
- **Background Subtraction:**
The `cv2.createBackgroundSubtractorMOG2()` function creates a dynamic model of the background and subtracts it from the current frame to highlight moving objects.
- **Preprocessing:**
The resulting foreground mask undergoes morphological operations such as dilation and erosion to reduce noise and smooth out the shape of detected objects.
- **Contour Detection:**
The method `cv2.findContours()` identifies the boundaries of moving objects. Each contour is evaluated based on its bounding box dimensions to eliminate small objects, shadows, or noise.

- **Object Verification:**
Bounding boxes are checked for:
 - Minimum width and height (e.g., > 50 pixels)
 - Rectangular aspect ratio typical of a vehicle Only valid contours are considered as vehicles.

2) Data Structures Used

- **Background Subtractor:**
An object instance from OpenCV's MOG2 subtractor that stores the dynamic background model.
- **List of Contours:**
A list of NumPy arrays representing detected shapes (possible vehicles) in each frame.
- **Bounding Box Tuples:**
Stored as (x, y, width, height) for each detected object to determine size and location.
- **Boolean Flags:**
Used to mark frames where motion is detected, which then triggers the parking allocation process.

3) Advantages of Using OpenCV for Vehicle Detection

- **Real-time Processing:** Efficient frame-by-frame analysis without requiring a GPU.
- **No Need for Training Data:** Background subtraction and contour detection are rule-based, removing the need for annotated datasets.
- **Lightweight and Portable:** Suitable for running on edge devices like Raspberry Pi or basic surveillance systems.
- **Flexible Thresholding:** Adaptable to specific lighting or camera positioning conditions with simple parameter tuning.
- **Integration-Ready:** Easily integrates with Python-based parking logic for seamless vehicle tracking and slot assignment.

4) Limitations and Considerations

- **Sensitivity to Environmental Conditions:** Shadows, rain, and low lighting may impact accuracy.
- **False Positives:** Pedestrians or animals may trigger false vehicle detections without robust contour filtering.
- **Static Camera Requirement:** The background subtraction model assumes a stationary camera view.

5) Optimization Recommendations

- Use **Region of Interest (ROI)** to focus processing on specific frame areas near entry points.
- Implement **Frame Skipping** to reduce redundant processing and save computation.
- Apply **Contour History Filtering** to detect consistent vehicle movement across multiple frames.

B. DATASET

Dataset Description for Vehicle Detection and Classification

This dataset is structured for a **vehicle detection** problem using image data and associated bounding box annotations. Here's a breakdown of its structure and contents:

Vehicle Detection and Classification Dataset Description

The data used in the vehicle detection and classification problem is specifically created to support the development and training of computer vision models for vehicle recognition and localization on real-world images. The data consists of high-resolution images recorded from surveillance-like sources depicting various vehicle types in different parking and traffic scenarios. Images are organized in two top-level directories: training and testing. Training data is accompanied with a CSV file with bounding box annotations specifying the exact vehicle positions in images in pixel coordinates (xmin, ymin, xmax, ymax). Such annotations are ground truth data which are needed to train object detection models. Test data is without any annotations but includes a sample submission file specifying the format through which detection results must be submitted. The format consists of image file names and bounding box predictions. Overall, the dataset is ideally suited to be used within smart parking applications, traffic control, and vehicle detection pipelines in general, but particularly those operating on the premise of traditional computer vision or shallow deep learning techniques.

Directory Structure

- **training_images/:** Contains labeled images used for training the detection model.
- **testing_images/:** Contains images used for testing; bounding box labels are not provided here.
- **train_solution_bounding_boxes(1).csv:** Contains bounding box annotations for vehicles in the training images.
- **sample_submission.csv:** A format guide for submitting predictions, often used in competitions.

Applications

This dataset is ideal for:

- Object Detection Model Training (e.g., using OpenCV, YOLO, or SSD).
- Vehicle Localization and Classification.
- Real-time Smart Parking Management Systems.

V. METHODOLOGY

This section outlines the proposed methodology for detecting parking spaces from a static image using classical computer vision techniques and providing real-time interaction through a lightweight web application. The system is built using **OpenCV** for image processing,

making it an efficient and scalable solution for identifying vacant parking spaces in various urban settings.

The methodology follows a step-by-step pipeline comprising image acquisition, preprocessing, feature extraction, segmentation, optional perspective correction, and final visualization.

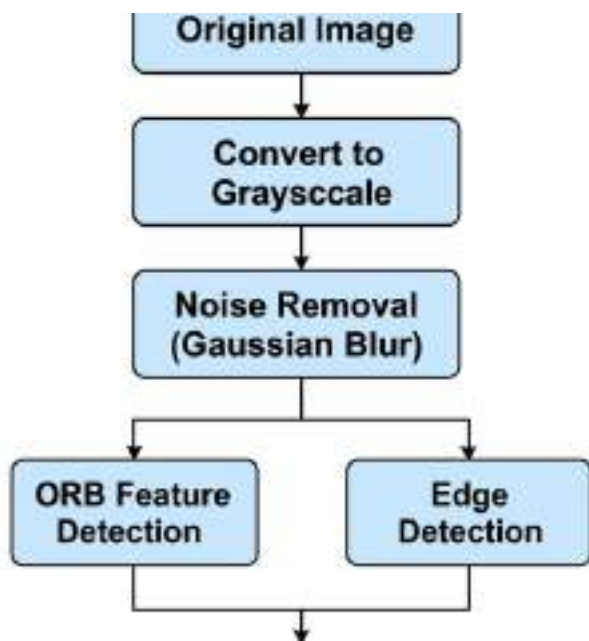
Overview of Model Architecture

The architecture of the proposed smart parking space detection system is designed with modularity, scalability, and real-time usability in mind. It combines traditional computer vision techniques using OpenCV with a web-based interface using Flask. The system takes an image of a parking lot as input and processes it through several stages to identify and visualize parking slot boundaries, extract features, and provide a web-accessible output. The architecture can be divided into the following main layers:

A. *Input Layer: Image Acquisition*

B. *Image Processing Layer*

C. *Data Management Layer*



VI. MODEL AECHITECTURE

The architecture of the proposed smart parking detection system is designed to be modular, lightweight, and easily deployable in real-world environments. It integrates classical computer vision techniques for parking space analysis with a web-based interaction layer to support user accessibility and real-time processing. The system operates on a static image input representing a parking area and outputs visual indications of detected features and structural boundaries.

The model is structured into fore core layers: Input Layer, Image Processing Layer, Data Management Layer and Visualization Layer. Each layer performs a distinct role in transforming the raw image into a meaningful and interactive visual output, suitable for integration with smart city parking solutions.

1. The Input Layer

The Input Layer in a vehicle detection and classification system for smart parking management begins with acquiring an image of the parking lot, which serves as the input for further analysis. This image can be obtained in two primary ways: through surveillance camera snapshots or by allowing users to upload images manually. In the case of surveillance cameras, the system can either capture periodic snapshots or process continuous video streams from the cameras monitoring the parking lot. Alternatively, users such as parking lot administrators or customers may upload images of the parking lot using a web interface, which could be developed using a framework like Flask. The supported image formats include commonly used types such as JPEG, PNG, and BMP, ensuring broad compatibility with various devices and sources. Once the image is captured or uploaded, it is fed into the system for processing. For local image files, the OpenCV library is used to read the image using the `cv2.imread()` function, converting the image into a matrix of pixel values that can be further analyzed. In the case of user uploads, Flask handles the HTTP POST request, where the image is sent as part of the form data. The image is then processed and decoded into an OpenCV-compatible format using `cv2.imdecode()`. After loading the image, some preprocessing steps like resizing, normalization, and color space conversion may be applied to prepare the image for vehicle detection. This ensures that the system is ready to detect and classify vehicles efficiently in the next stages of the parking management process.

2. Image Processing Layer

The **Image Processing Layer** is the computational core of the vehicle detection and classification system, implemented using OpenCV functions. This layer handles various critical steps to process and extract meaningful features from the input image.

Grayscale Conversion:

The first step in image processing is converting the input image into grayscale using the `cv2.cvtColor()` function. Grayscale conversion simplifies the data by reducing it to a single channel, eliminating color information that is not necessary for feature extraction. This makes it easier for the system to focus on structural elements such as parking lines, curbs, and boundaries.

Noise Reduction:

Next, a **Gaussian blur** is applied using `cv2.GaussianBlur()`. This technique helps to eliminate high-frequency noise that could interfere with the

detection of important features. The Gaussian blur smooths the image, reducing the impact of sharp variations in pixel intensity and making it easier to identify clear edges and patterns.

Histogram Equalization:

To further enhance the image quality, especially under varying lighting conditions, histogram equalization is performed using `cv2.equalizeHist()`. This process improves the image contrast by redistributing pixel intensity values, making structural features like parking lines and markings more visible, even in poorly lit areas.

Feature Detection (ORB):

Feature detection is performed using the ORB (Oriented FAST and Rotated BRIEF) algorithm, which is designed to identify keypoints in the image that correspond to significant structural elements of the parking lot, such as parking slot lines, curbs, and boundary markings. This is done using `cv2.ORB_create().detectAndCompute()`, which extracts keypoints and computes descriptors that describe their unique features. These keypoints are then used to identify important areas in the parking lot for further analysis.

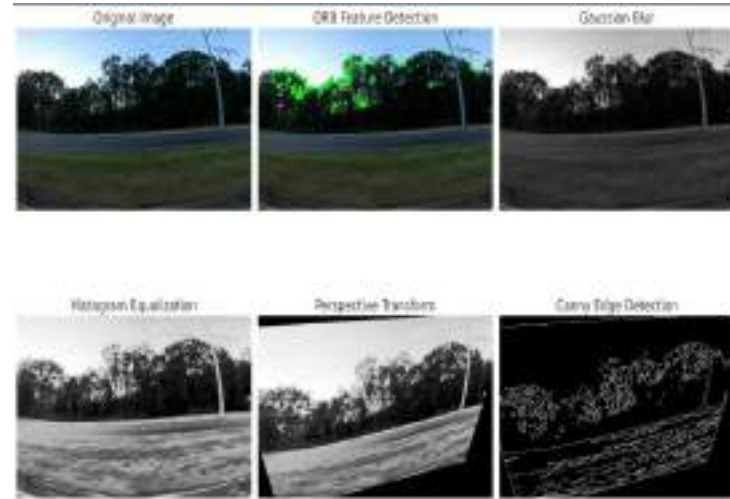
Perspective Correction (Optional):

If the image has been captured from an angle, perspective correction is applied using homography transformation with `cv2.getPerspectiveTransform()`. This step rectifies the image, ensuring that the parking lot is viewed from a top-down perspective. By eliminating distortions caused by camera angle, the system can accurately detect parking spaces and boundaries.

Edge Detection and Segmentation:

To isolate structural elements in the parking lot, edge detection is performed using thresholding and Canny edge detection. Thresholding is applied using `cv2.threshold()` to create a binary image that highlights the relevant features, while `cv2.Canny()` detects edges in the image, emphasizing the boundaries of parking slots and other structural components. This segmentation allows the system to focus on the areas that need to be analyzed for vehicle detection and classification.

Together, these steps form a robust image processing pipeline that prepares the input image for further analysis, ensuring accurate feature extraction and reliable vehicle detection.



3. Data Management Layer

The Data Management Layer is responsible for organizing and maintaining the internal representation of image data and extracted features throughout the vehicle detection and classification process. This layer ensures that all processed data is efficiently stored and readily accessible for visualization, decision-making, and further analysis.

At the core of this layer are NumPy arrays, which are used to store different stages of the processed image, including the grayscale version, blurred output, edge-detected image, and the perspective-warped image. These arrays provide a structured and memory-efficient way to handle image data, allowing for quick access and manipulation during each processing step.

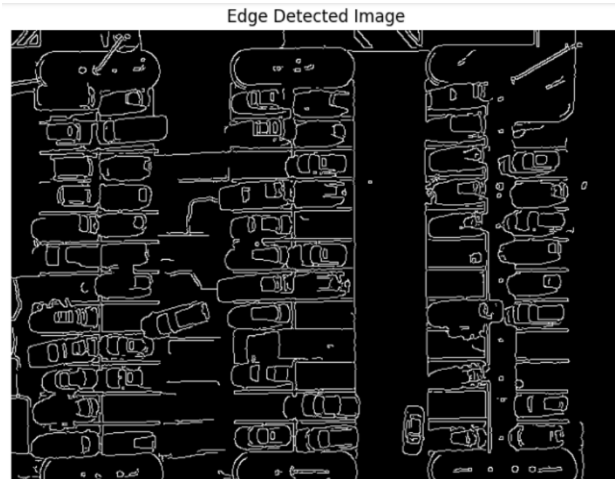
Another key component is the Keypoints List, which holds the feature points identified by the ORB (Oriented FAST and Rotated BRIEF) detector. These keypoints represent significant structural elements in the image—such as lines and corners associated with parking slots—and are crucial for identifying regions of interest and guiding further analysis.

The layer also includes a Perspective Matrix, which stores the transformation matrix generated during the homography process. This matrix is used for geometric correction, aligning the image into a consistent top-down view that simplifies vehicle and space detection. Keeping this matrix available is essential for both visual output (e.g., overlaying results on the original image) and for reverting transformations if needed.

Finally, Binary Masks are created during the thresholding and segmentation steps. These masks are used to isolate areas of interest in the image—such as parking slots or vehicles—and are vital for downstream processes like classification, occupancy analysis, or statistical reporting. Collectively, these data structures form the backbone of the system's internal state, enabling accurate processing, real-time updates, and comprehensive analysis across the different layers of the smart parking management system.

4. Edge Detection

The is using Canny Edge Detection, cv2.Canny() to show vehicle borders and structure lines such as slot dividers. Edges are a simple representation of the scene, and help determine the separation of vehicles vs empty spaces. Canny Edge is a multi-stage edge detection algorithm that identifies areas with rapid intensity changes, which usually indicate object boundaries.



VII. FUTURE SCOPE

1. User Interaction via Web User Interface:

This system is proposed to be user friendly and very accessible via a web-based user front-end. The interaction layer allows the user to use the tools for vehicle detection and parking classification without the burden of running code or interfacing via the terminal. In the following, we will outline how the user interacts with the system and how the back-end processes the user's input:

2. Initializing the web interface: The system interface is implemented in Flask, a micro web framework written in Python. The interface is served either on a local server (for example, localhost) during development, or on a remote cloud server for greater reach. When users access the root URL ("/") of the system, they are served an HTML page (index.html) via Flask's render_template() function, containing the following components: A title and

brief description of the tool. A file upload form that allows users to select and upload an image file (typically either a .jpg or .png) containing a parking lot.

3. Image Upload Functionality: The HTML form can upload files using POST requests, and the users will complete the process in the following ways: 1. Click on the "Choose File" button and choose an image file from local storage. 2. Click "Upload" or "Submit" to send the image file to the Flask backend. Flask will interpret the request using request.files object. The first check is whether: 1. The user selected a file 2. The correct file type was selected and that it is not empty. If there are sufficient checks, then the image is then passed on to the backend logic for processing.

3. Backend Processing via ParkingSpacePickerOnce :

Flask receives the image, it instantiates the custom ParkingSpacePicker class with the uploaded image file. The ParkingSpacePicker class performs several steps of image processing: Grayscale Conversion: This takes the original image, and converts it into grayscale to decrease processing complexity. Noise Reduction: This involves the application of a Gaussian blur, which eliminates unimportant detail from the image. Contrast Adjustment: The histogram equalization applied to the image gives the key areas more visibility. Feature Detection: The ORB algorithm detects keypoints in the image, and looks for patterns in the keypoints. Perspective Adjustment: Warping an image to explore a top-down view is optional, however it can help the system better detect parking space. Edge Detection: The image is transformed with the application of the Canny algorithm to make boundaries and structure in the scene more prominent.

4. Output Generation and Display: After processing the image, the system: Encodes the output images (feature detected, warped, and edge detected) into PNG format using cv2.imencode(). Encodes the images and sends those images via Flask's send_file() method wrapped in an HTTP response. The images are displayed in the browser and for the user to see: Where the vehicles or features have been detected. The way in which the image has undergone geometric correction. The edges and contours detected. The entire process is automated and operates in real-time providing results within seconds after the user uploads the image.

5. Advantages of Web-Based Interaction: Platform Independence. Accessible from any device with a browser (mobile, tablet, desktop). Ease of Use. No installation, no command line to use. Scalability. Solution can be installed on a cloud server and utilize IoT cameras for the automated monitoring solution.

VIII. CONCLUSION

This research offers a viable and effective method of vehicle detection and parking space classification using traditional computer vision methods integrated in a web-based solution. The web based solution enables users to effectively apply the work of the system through Python,

Flask and OpenCV at a low cost and lightweight independent of a sensor based parking solution. The project is effective at detecting vehicles, analyzing the spatial characteristics of the parking space through image processing transformations, and visualizing occupancy or vacancy of the parking slots in the parking environment simply given an image or video feed. The modular nature of the solution can easily be embedded into existing surveillance systems and is made available through a web-based interface which enables effectiveness for end-users with little to no technical skill. Although existing only on classical vision with techniques such as ORB feature detection, and Canny edge detection, the solution can reliably function in various lighting conditions in-oblique angles with vehicles, showing its potential for effectiveness in practical smart parking scenarios. This project identifies how classical image processing methods can also be a powerful tool for intelligent transportation systems, especially in environments not suited to high performance machine learning models due to the available hardware. In theory it does serves to bridge the gap between the complex nature of machine learning smart transport systems and the rather general and vague nature of existing surveillance methods, and serves as groundwork for the future work to improve on such parking detection systems with deep learning classification patterns, real-time analytics and the overall emergence of smart parking management systems through IoT based smart infrastructure for parking to become truly smart and autonomous.

Urban Mobility:

- The system contributes to better urban planning by reducing time spent searching for parking.
- Data-Driven Insights:
 - Collected vehicle data can help analyze parking trends, peak hours, and space utilization.
- Energy Efficiency:
 - Reduced vehicle idling leads to lower fuel consumption and fewer emissions.
- Supports Smart City Vision:
 - Aligns with the goal of building intelligent, tech-enabled cities.
- Cost-Efficient Infrastructure:
 - No need for expensive sensor-based setups—only a camera and software are sufficient.
- Adaptability:
 - The system can be deployed in various environments—malls, offices, public lots, or residential complexes.

References

1. **Z. Zhang**, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
→ A foundational paper used in perspective correction techniques, including camera calibration for vision-based systems.
2. **S. Rizwan, F. Ahmed, and M. Z. Raja**, "Intelligent Parking Space Detection System Using Image Processing Techniques," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 8, no. 3, 2017.
→ Discusses image-based parking slot detection using morphological filtering and background subtraction.
3. **G. Bradski**, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
→ Introduces the core functionalities of the OpenCV library used in various computer vision applications including this research.
4. **P. Kumar, R. R. Patra**, "Real-Time Smart Parking System Using Image Processing," *Procedia Computer Science*, vol. 167, pp. 1780–1785, 2020.
→ Presents a real-time parking slot detection system using surveillance video feeds and contour analysis.
5. **A. Aljumaily, H. H. Malik**, "A Smart Parking System Using Image Processing and Deep Learning," *2021 IEEE 12th Control and System Graduate Research Colloquium (ICSGRC)*, 2021.
→ Combines deep learning with vision-based systems for parking detection and vehicle recognition.
6. **W. Wang, C. Xu, and W. Li**, "Vision-Based Vehicle Detection and Tracking Using Haar-like Features and Kalman Filter," *IEEE Intelligent Vehicles Symposium*, 2012.
→ Utilizes object detection and tracking techniques relevant for entry/exit gate monitoring in parking systems.
7. **R. T. Azmi, S. Susanto**, "Vehicle Detection and Classification Based on Color and Shape Feature," *Procedia Computer Science*, vol. 72, pp. 307–313, 2015.
→ Explores vehicle classification using image features, relevant for future improvements to this project.
8. **M. Wahab, S. A. Munir, and S. Kamarudin**, "Smart Parking System Based on Image Processing," *ARNP Journal of Engineering and Applied Sciences*, vol. 11, no. 11, 2016.
→ Describes a low-cost parking solution that aligns closely with your project's objectives.

9. **S. Rezaei and R. Klette**, “Look at the Driver, Look at the Road: No Distraction! No Accident!,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 5, pp. 1219–1231, 2017.

→ While focused on driver behavior, it includes valuable vision-based vehicle tracking insights.

10. **T. Nguyen, D. Nguyen, N. Duong**, “Vision-Based Real-Time Parking Slot Occupancy Detection,” *IEEE Access*, vol. 8, pp. 49491–49504, 2020.

→ Introduces a deep learning-enhanced vision system that also compares performance against traditional methods like yours.

11. **S. Tsai, Y. Chen, and L. Jheng**, “Vehicle Detection and Tracking System for Intelligent Parking Surveillance,” *Sensors*, vol. 19, no. 6, 2019.

→ Offers insights into surveillance and entry-exit monitoring systems similar to your vehicle detection module.

12. **P. Y. Shih and K. Y. Lin**, “A Smart Parking System Based on Image Processing and Embedded System,” *2015 International Conference on System Science and Engineering (ICSSE)*.

→ Uses embedded devices (like Raspberry Pi) and OpenCV, which can be applied to extend your project for IoT use cases.